1407 comment and documenting

```
import { auth, db } from "../config.js";
import { createUserWithEmailAndPassword, updateProfile } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.js";
import { doc, setDoc } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js";

const signupForm = document.getElementById("interactive-form");

if (signupForm) {
  signupForm.addEventListener("submit", async function (event) {
    event.preventDefault();

    // FIX: Match your HTML IDs - use nameInput not name
    const name = document.getElementById("name")?.value.trim() || document.getElementById("nameInput")?.value.trim();
    const email = document.getElementById("email")?.value.trim() || document.getElementById("emailInput")?.value.trim();
    const password = document.getElementById("password")?.value || document.getElementById("passwordInput")?.value;
    const confirmPassword = document.getElementById("confirmpassword")?.value;
    const termsConsent = document.getElementById("terms-consent")?.checked;
    const feedbackContainer = document.getElementById("form-feedback");

    if (feedbackContainer) {
      feedbackContainer.textContent = "";
      feedbackContainer.style.color = "inherit";
    }

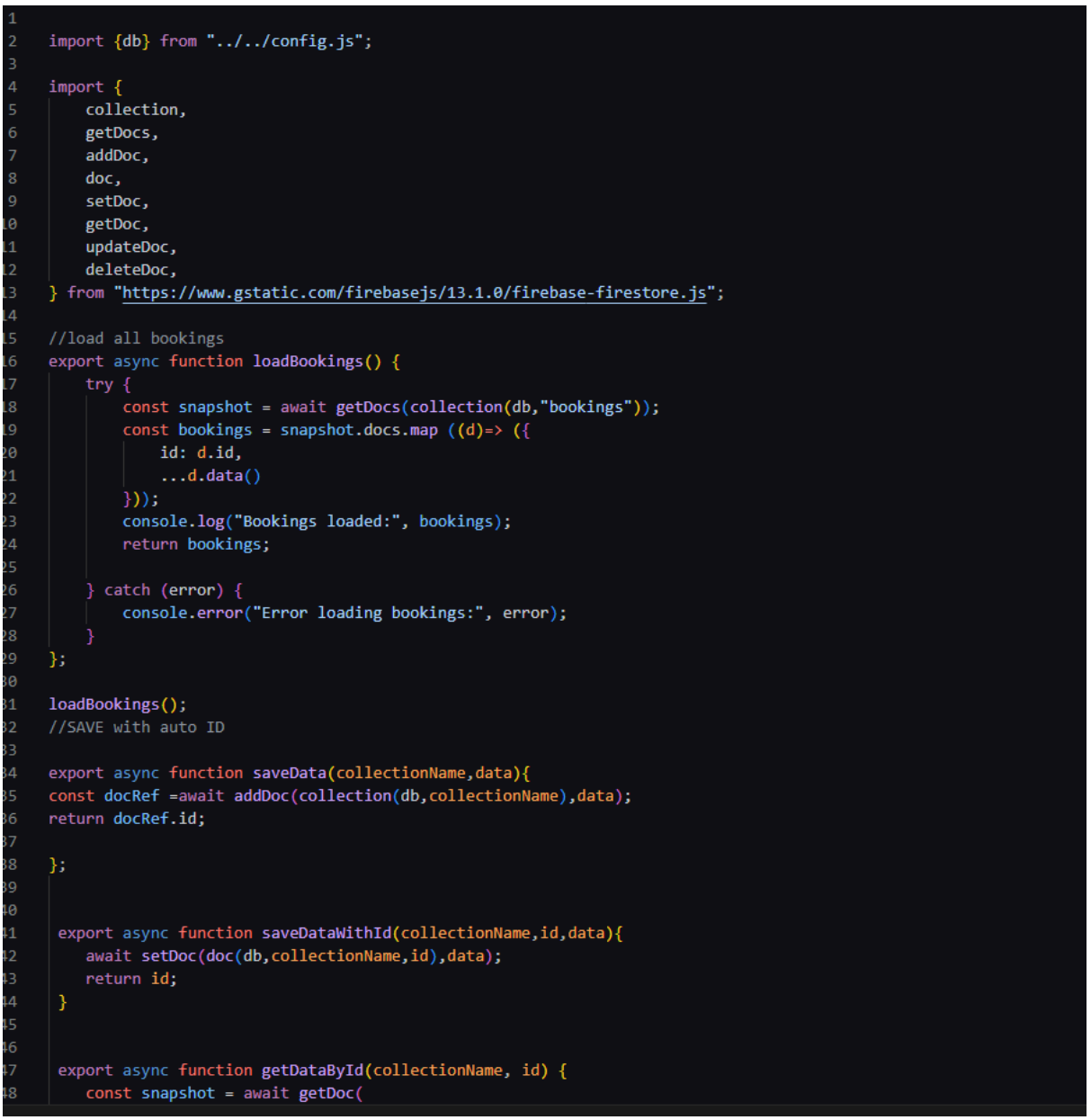
    // 1. Password Matching Validation (YOUR CODE - GOOD)
    if (password !== confirmPassword) {
      if (feedbackContainer) {
        feedbackContainer.style.color = "#dc3545";
        feedbackContainer.textContent = "❌ Passwords do not match. Please try again.";
      } else {
        alert("Passwords do not match");
      }
      return;
    }

    // 2. Minimum Length Rule Check (YOUR CODE - GOOD)
    if (password.length < 6) {
      if (feedbackContainer) {
        feedbackContainer.style.color = "#dc3545";
        feedbackContainer.textContent = "❌ Password must be at least 6 characters long.";
      }
      return;
    }

    // 3. Document Terms Agreement Safeguard (YOUR CODE - GOOD)
    if (termsConsent !== undefined && !termsConsent) {
      if (feedbackContainer) {
        feedbackContainer.style.color = "#dc3545";
        feedbackContainer.textContent = "❌ You must agree to the Terms & Conditions.";
```

```
1
2   import { initializeApp } from 'https://www.gstatic.com/firebasejs/13.1.0/firebase-app.js';
3   import { getAuth } from 'https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.js';
4   import { getFirestore } from 'https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js';
5
6
7   // For Firebase JS SDK v7.20.0 and later, measurementId is optional
8   const firebaseConfig = {
9     apiKey: "AIzaSyAkmGzJ17_YmTgLRZ_ThEep6sg9gxu7h-c",
0     authDomain: "hotel-booking-2bb67.firebaseapp.com",
1     projectId: "hotel-booking-2bb67",
2     storageBucket: "hotel-booking-2bb67.firebasestorage.app",
3     messagingSenderId: "158489814433",
4     appId: "1:158489814433:web:11c989e20b23a084bad7e0",
5     measurementId: "G-84PXRD32TF"
6   };
7
8   const app = initializeApp(firebaseConfig);
9   export const auth = getAuth(app);
0   export const db = getFirestore(app);
1   export { firebaseConfig };
```

1407 comment and documenting



```
1
2   import {db} from "../../config.js";
3
4   import {
5       collection,
6       getDocs,
7       addDoc,
8       doc,
9       setDoc,
10      getDoc,
11      updateDoc,
12      deleteDoc,
13  } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js";
14
15  //load all bookings
16  export async function loadBookings() {
17      try {
18          const snapshot = await getDocs(collection(db,"bookings"));
19          const bookings = snapshot.docs.map ((d)=> ({
20              id: d.id,
21              ...d.data()
22          }));
23          console.log("Bookings loaded:", bookings);
24          return bookings;
25
26      } catch (error) {
27          console.error("Error loading bookings:", error);
28      }
29  };
30
31  loadBookings();
32  //SAVE with auto ID
33
34  export async function saveData(collectionName,data){
35  const docRef =await addDoc(collection(db,collectionName),data);
36  return docRef.id;
37
38  };
39
40
41   export async function saveDataWithId(collectionName,id,data){
42      await setDoc(doc(db,collectionName,id),data);
43      return id;
44   }
45
46
47   export async function getDataById(collectionName, id) {
48      const snapshot = await getDoc(
```

1407 comment and documenting

```
let currentRoom = null;

document.querySelectorAll(".view-room-btn, button").forEach(btn => {
  if(btn.textContent.includes("View room")){
    btn.classList.add("view-room-btn");
    btn.addEventListener("click", () => {
      const name = btn.dataset.name || btn.closest(".card, [class*=card]")?.querySelector("h3, h2")?.textContent?.trim() || "Monastero Sa
      const room = roomsData[name] || roomsData["Monastero Santa Rosa Hotel & Spa"];
      currentRoom = room;

      // Fill modal
      document.getElementById("modalImage").src = room.image;
      document.getElementById("modalRoomName").textContent = room.roomName;
      document.getElementById("modalPlace").textContent = room.place;
      document.getElementById("modalCategory").textContent = room.category;
      document.getElementById("modalCategory2").textContent = room.category;
      document.getElementById("modalBed").textContent = room.bed;
      document.getElementById("modalBase").textContent = "R" + room.base;
      document.getElementById("modalSurge").textContent = "R" + room.surge;
      document.getElementById("modalInventory").textContent = room.inventory;
      document.getElementById("modalState").textContent = room.state;
      document.getElementById("modalStatus").textContent = room.inventory;
      document.getElementById("modalStatus").style.background = room.statusColor;
      document.getElementById("modalDesc").textContent = room.desc;
      document.getElementById("modalPrice").textContent = "R" + room.price + " / night";

      document.getElementById("roomModal").style.display = "flex";
    });
  }
});

document.getElementById("closeModal").onclick = () => {
  document.getElementById("roomModal").style.display = "none";
};

// Book from modal -> Save to Firestore
document.getElementById("modalBookBtn").onclick = async () => {
  if(!currentRoom) return;
  const booking = {
    place: currentRoom.roomName,
    name: currentRoom.place,
    category: currentRoom.category,
    bed: currentRoom.bed,
    price: currentRoom.price,
    basePrice: currentRoom.base,
```

1407 comment and documenting

```
import { auth} from "../config.js";
import {signInWithEmailAndPassword } from 'https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.js';



document.addEventListener("DOMContentLoaded", () => {
  // Use IDs — change these to match your inputs
  const nameInput = document.getElementById("name") || document.querySelector('input[placeholder*="Name"]');
  const emailInput = document.getElementById("email") || document.querySelector('input[type="email"]');
  const passwordInput = document.getElementById("password") || document.querySelector('input[type="password"]');
  const loginBtn = document.getElementById("loginBtn") || document.querySelector('button');

  console.log("Found:", { loginBtn, emailInput, passwordInput });

  if (!loginBtn) {
    console.error("Login button not found! Add id='loginBtn' to your button");
    return;
  }

  loginBtn.addEventListener("click", async (e) => {
    e.preventDefault();

    const email = emailInput?.value.trim();
    const password = passwordInput?.value.trim();
    const name = nameInput?.value.trim() || "otlotleng";

    if (!email || !password) {
      alert("Enter email and password");
      return;
    }

    loginBtn.textContent = "Logging in...";
    loginBtn.disabled = true;

    try {
      const cred = await signInWithEmailAndPassword(auth, email, password);

      localStorage.setItem("user", JSON.stringify({
        uid: cred.user.uid,
        email: cred.user.email,
        name: name
      }));

    localStorage.setItem("userEmail", cred.user.email);
  localStorage.setItem("userId", cred.user.uid);
  localStorage.setItem("isLoggedIn", "true");

      alert(`Welcome ${name} !`);
```